

Übung 4: Wartbare Projekte, manuelle Eingriffe und If-Logik

Kontrollwerte

Das Skript meldet am Ende diese Kontrollwerte:

- Eingelesene Datenzeilen: **280**
- Bei der Plausibilitätsprüfung auffällige Zeilen (Schwellen 1,20 / 0,83): **13**
- Beispiel-Overrides in `data/overrides.csv`: **3**
- Tatsächlich geänderte Zeilen (Override greift auf vorhandenen Schlüssel): **3**
- Audit-Protokoll geschrieben nach: `data/audit/audit_overrides.csv`

Arbeitsumgebung

- RStudio-Projekt: `projekt/` (Datei `vgr-r-schulung.Rproj`). Arbeitsverzeichnis = dieser Ordner.
- Daten liegen unter `data/`. Greifen Sie **immer** über `here::here("data", ...)` zu.
- Startdatei: `R/lab4_projekt_overrides.R` (enthält # TODO-Lücken).

Ausführung aus dem Projektordner:

```
Rscript R/lab4_projekt_overrides.R
```

Projektpfad

Damit `here::here()` zuverlässig den Projektordner findet, braucht das Projekt einen Anker. Im RStudio-Projekt genügt die `.Rproj`-Datei; auf der Kommandozeile nutzen wir zusätzlich `here::i_am(...)`.

```
library(here)
here::i_am("R/lab4_projekt_overrides.R")
here() # zeigt den Projektordner
```

Niemals `setwd("/absoluter/Pfad")` im Skript. Das bricht auf jedem anderen Rechner und in der automatisierten Produktion. `here::here()` macht den Pfad portabel.

Schritt 1: Daten robust einlesen (eigene Funktion mit Validierung)

Schreiben Sie eine Funktion `lade_lieferung(pfad)`, die

1. prüft, dass die Datei existiert (`stopifnot(file.exists(pfad))`),
2. die CSV mit `readr::read_csv()` einliest,
3. prüft, dass die erwarteten Spalten vorhanden sind und `wert` numerisch ist,
4. den eingelesenen Tibble zurückgibt.

```
lade_lieferung <- function(pfad) {
  # TODO: Datei prüfen, CSV einlesen und Eingaben validieren.
  NULL
}
```

Aufgabe 1a: Vervollständigen Sie `lade_lieferung()` in der Startdatei und lesen Sie `data/lieferung_2024q4.csv` ein.

Hinweis: `stopifnot()` akzeptiert benannte Ausdrücke. Der Name wird zur Fehlermeldung, wenn die Bedingung `FALSE` ist. Das ist die einfachste Form der Eingangsvalidierung für eine Funktion.

Schritt 2: Plausibilitätsregel mit `case_when()`

Die Wirtschaftsbereiche haben sehr unterschiedliche Größenordnungen (Bereich A ist ~9 Mrd., Bereich BE ~234 Mrd.). Ein globaler Schwellenwert taugt also nicht. Wir markieren auffällige Werte **relativ zum eigenen historischen Median je Bereich**.

Bilden Sie je `bereich_code` den Median über alle Quartale (`group_by + mutate(median(wert))`) und das Verhältnis `wert / median`. Die VGR-Reihen sind ruhig, daher gelten bereits Abweichungen von etwa ± 20 Prozent gegenüber dem eigenen Median als prüfbedürftig. Markieren Sie mit `dplyr::case_when()`:

```
markiere_plausibilitaet <- function(daten) {
  # TODO: Median je bereich_code und das Verhältnis wert/Median berechnen.
  # TODO: Mit case_when() als ungewöhnlich hoch, ungewöhnlich niedrig
  #       oder ok markieren.

  daten
}
```

Aufgabe 2a: Vervollständigen Sie die Funktion.

Aufgabe 2b: Ergänzen Sie eine zweite, vektorisierte Markierung mit `ifelse()`, die nur zwischen "prüfen" und "ok" unterscheidet (alles, was nicht "ok" ist, wird zu "prüfen"):

```
daten$pruefflag <- NULL # TODO: mit ifelse() erzeugen
```

`case_when()` wertet die Bedingungen **von oben nach unten** aus; die erste zutreffende gewinnt. Der Default-Zweig ist `TRUE ~ ...`. Vergessen Sie ihn nicht, sonst entstehen `NA`.

Schritt 3: Override-Tabelle anlegen und einlesen

Manuelle Korrekturen werden **nicht** direkt im Datensatz überschrieben, sondern in einer separaten, versionierbaren Tabelle dokumentiert: `data/overrides.csv`.

Spalten: `jahr`, `quartal`, `bereich_code`, `neuer_wert`, `bearbeiter`, `datum`, `grund`.

Falls die Datei noch nicht existiert, legt das Skript sie mit 2 bis 3 Beispiel-Overrides an (Funktion `stelle_overrides_bereit()`).

```
stelle_overrides_bereit <- function(pfad) {
  # TODO: Override-Tabelle anlegen und mit readr::write_csv() speichern.
  invisible(NULL)
}

overrides <- NULL # TODO: data/overrides.csv einlesen
```

Aufgabe 3a: Lassen Sie das Skript `data/overrides.csv` anlegen (falls nicht vorhanden) und lesen Sie sie ein.

Schritt 4: Overrides anwenden (`rows_update()`) und Audit markieren

Wenden Sie die Overrides per Schlüssel (`jahr`, `quartal`, `bereich_code`) an. `dplyr::rows_update()` aktualisiert genau die passenden Zeilen. Davor merken Sie sich den alten Wert, damit Sie Vorher/Nachher und das Audit erzeugen können.

```
wende_overrides_an <- function(daten, overrides) {
  # TODO: Patch aus Schlüsselspalten und neuer_wert erstellen.
  # TODO: Betroffene Zeilen vor der Änderung mit semi_join() sichern.
  # TODO: Overrides mit rows_update() anwenden und markieren.
  NULL
}
```

Aufgabe 4a: Vervollständigen Sie die Funktion. `unmatched = "ignore"` sorgt dafür, dass ein Override auf einen nicht vorhandenen Schlüssel nicht zum Fehler führt (sondern still ignoriert wird).

Aufgabe 4b: Erstellen Sie eine Vorher/Nachher-Tabelle, die den alten und den neuen Wert je geänderter Schlüssel nebeneinander zeigt.

`rows_update()` braucht eindeutige Schlüssel und passende Spaltentypen. Wenn die `overrides.csv` z. B. `jahr` als Text einliest, schlägt der Join fehl. Lesen Sie die Schlüsselspalten als Integer ein oder konvertieren Sie sie.

Schritt 5: Audit-Protokoll schreiben

Schreiben Sie eine Funktion `schreibe_audit()`, die je geänderter Zeile festhält: Schlüssel, alter Wert, neuer Wert, Differenz, Bearbeiter, Datum, Grund. Speichern Sie das Protokoll als `data/audit/audit_overrides.csv` (Ordner ggf. anlegen).

```
schreibe_audit <- function(vorher, overrides, ziel) {
  # TODO: Audit-Tabelle erstellen, Zielordner anlegen und CSV schreiben.
  invisible(NULL)
}
```

Aufgabe 5a: Erzeugen Sie das Audit aus der Override-Tabelle und dem alten Wert und schreiben Sie es per `readr::write_csv()`.

Aufgabe 5b: Geben Sie am Ende die Kennzahlen aus (eingelene Zeilen, Anzahl Overrides, Anzahl tatsächlich geänderter Zeilen). Vergleichen Sie mit den erwarteten Ergebnissen oben.

Weiterführend (optional)

- Erweitern Sie die Override-Tabelle um eine Spalte `gueltig_ab` und wenden Sie nur Overrides an, deren Datum in der Vergangenheit liegt (`Sys.Date()`).
- Schreiben Sie einen kleinen `testthat`-Test für `lade_lieferung()` (z. B. Fehler bei fehlender Datei).